SPECS · PRODUCT TAXONOMY, GRAPHQL & CPG PLANNING

Product Taxonomy, GraphQL and the GID Rename

Shopify moved product identity into a **typed graph** with **GID-form identifiers** and server-side events. What that changes for the item master, the parent/child model, and every downstream join — and which five identifiers a CPG item master has to carry.

CPG planning toward Sunrise 2027

FOR: ANYONE WHO OWNS PRODUCT DATA, FULFILMENT, OR RETURNS ACROSS MORE THAN ONE MARKET. **NOT:** A PACKAGING STORY. THE BARCODE IS WHERE THE THREAD BECOMES VISIBLE, NOT WHERE IT STARTS.

The misconception, first

A QR code on a package is not a Sunrise 2027 QR code.

For a checkout to scan it, it has to be a **GS1 Digital Link** carrying a valid GTIN — a URI of the form `https://yourdomain.com/01/09506000134352`. A marketing QR that points at a landing page will work fine on a phone and fail completely at the till.

That distinction is most of the confusion in the market right now, and it costs artwork revisions to discover late.

What Sunrise 2027 actually is

GS1's target is that retailers globally are **able to scan 2D barcodes at point of sale by the end of 2027**. It is a retailer capability date, which makes it a brand readiness date.

Two things it is not:

- **It is not a switch-off.** Linear barcodes keep working. The transition is coexistence, not replacement, and packs will carry both for years.
- **It is not automatic.** Nothing about your existing UPC becomes a Digital Link by itself.

Reference: GS1 2D barcodes · GS1 Digital Link

The numbers, and which of them is real

Four identifiers get used interchangeably in conversation and are not interchangeable at all.

	ASSIGNED BY	UNIQUE WHERE	PORTABLE	SCANNABLE AT POS
GTIN	GS1 registry	Globally	✓	✓
MPN	You	Within your brand	Partly	×
SKU	You	Your systems only	×	×
ASIN	Amazon	Amazon	×	×

GTIN comes in four widths — GTIN-8, GTIN-12 (UPC-A), GTIN-13 (EAN-13) and GTIN-14 (cases). Fourteen digits is the canonical **field** width: GS1 specifies that all GTINs are right-justified with leading zeros into a 14-digit field for

processing, so a GTIN-13 stored correctly is `05901234123457`. There is no 16-digit GTIN. The 18-digit key in the family is the SSCC, for shipping containers.

A GTIN-13 decomposes as:

```
5 901234 12345 7
| |      |     └── check digit    (1)  computed
| |      └──────── item reference (5)  you assign
| └─────────────── company prefix (6)  GS1 assigns
└───────────────── from your licence
```

The check digit is arithmetic you can verify yourself — alternate ×1 and ×3 across the first twelve digits, sum, subtract from the next multiple of ten:

```
5(1) 9(3) 0(1) 1(3) 2(1) 3(3) 4(1) 1(3) 2(1) 3(3) 4(1) 5(3)

= 5+27+0+3+2+9+4+3+2+9+4+15 = 83   →   90 − 83 = 7 ✓
```

MPN is self-assigned, unregistered, has no check digit and no format. It is unique only within your brand, which is why Google requires *brand + MPN together* when a GTIN is absent. **SKU** is narrower still — merchant-internal, never an external identity. **ASIN** is Amazon's, exists inside Amazon, and stops at their boundary.

The gate is the registry, not the format

This is the part teams get wrong, and it is expensive.

A well-formed number is not a valid GTIN. The check digit proves the number is *structurally* correct; only the GS1 registry proves it is **yours**. Amazon validates listed GTINs against GS1's records rather than accepting anything that passes

the arithmetic — which is how a large population of sellers discovered that the cheap resold barcodes they had bought resolved to somebody else's company prefix. The listings were suppressed and the remedy was a real licence.

So the first step is commercial, not technical: a GS1 company prefix, from which item references and therefore GTINs are assigned. No membership, no GTIN, no Digital Link. (Get barcodes)

Budget the lead time. It is the longest pole in the project and it has nothing to do with engineering.

If you sell through retail, you already have them

The section above is written for a brand starting from nothing. Most enterprises are not, and their problem is more awkward.

Walmart, Target and Amazon all require a GTIN for item setup. So any brand with big-box distribution already holds GS1 licences and real, correct, in-use GTINs. The identifier is not missing.

It is just not *yours to answer with*. It was mapped during onboarding inside a channel management platform — one mapping per retailer, alongside each retailer's own item spec — and it never came back into the brand's own systems as authoritative. The number exists, is correct, is in daily use, and the brand cannot produce it without asking a vendor.

> **You already have the identifier. You are renting the only place it is joined.**

That is a different failure from not having one, and a more consequential one: when the platform rebrands, when a contract ends, or when a retailer changes spec, the mapping does not travel with you.

The reason it ended up there, which is not stupidity

Worth stating plainly, because the usual telling is unfair and the accurate version is more useful.

Brands with big-box distribution did not build direct-to-consumer capability, and that was a defensible commercial position for years — selling direct competes with the retailer carrying the volume. Under-investing in DTC was a rational answer to channel conflict, not an oversight.

The consequence was structural rather than technical. No DTC team meant nobody internal owning customer data, consent, catalog truth, or the operating knowledge that only comes from running a storefront. All of it was rented: the channel manager for syndication, an agency for the site, a consultancy for the integration. And each outsourced function removed the people who would have understood the next one, so the gap compounded instead of holding steady.

The requirements have now moved to precisely the layer that was never staffed. Event-time consent, per-market obligations, identifier authority, agent mandates — none of it is outsourceable, because it is continuous judgment about your own business. A retailer relationship can be managed by a partner. A consent decision at 14:32 cannot.

So the enterprise position today is: the identifiers exist, the volume exists, the brand exists, and there is no one inside who operates the layer that now matters. That is not a technology gap. It is a staffing decision made fifteen years ago, arriving on a deadline.

Which also explains why buying a fifth vendor does not close it. A capability gap created by purchasing cannot be purchased shut.

The thread these numbers already carry

Here is the part that makes this operational rather than cosmetic. The same identifiers are already load-bearing across the physical estate, and nobody thinks of them as one thing:

- **MPN → the factory.** The part being built, the BOM line, the firmware revision bound to it. Pick-and-pack runs off it.
- **Barcode → the WMS.** Receiving, putaway, shelf location, pick, pack, ship. Every scan in the building is that number.
- **The same code → fulfilment**, onto the carrier label, and then back through **RMA**, warranty scope and disposition.
- **The same code → commerce**, into Merchant Center, Shopping, social catalogs, and the conversion record.

GS1 already has the syntax for the part most teams hand-roll. A Digital Link carries more than the product:

```
/01/<gtin>/10/<batch>/21/<serial>

   01 = product      10 = batch / lot      21 = serial
```

That is how you get from *this product* to *this unit* — which is the difference between "we ship a device with that vulnerability" and "these 4,200 units, in these shipments, to these customers."

One thread: factory → warehouse → sale → return → recall. It exists whether or not anyone models it. The question is only whether it is one identifier or four reconciliations.

Where the thread breaks: the border

A return is the hardest test in the estate, because it exercises every per-market fact simultaneously — and it does so *after* the margin is gone.

A US return is one reverse lane, one tax treatment, one policy you wrote yourself. Cross-border, each of those splinters:

- **Withdrawal is a right, not a policy.** The EU's 14-day withdrawal right is granted to the buyer, and the legal guarantee runs two years, regardless of what your terms page says.
- **Customs runs backwards.** Duty and import VAT already paid need relief or refund; the unit has to import into wherever it is going. Wrong paperwork and the return costs more than the item.
- **Merchant of record decides who may accept it.** If the seller in that market is a separate entity or a marketplace, the return is theirs.
- **Disposition is regulatory, not logistical.** A unit returned in the EU may not be resellable in the US and vice versa — different marks, different radio approvals, different firmware locale. "Restock" is a compliance decision.
- **Refund rail and currency** must match the original capture.

Every one of those is a **per-market fact**. And most systems model market as a *display preference* — a currency symbol, a language, perhaps a tax rate. So RMA logic ends up hard-coded per country, in whichever system was nearest, by whoever was on that project. That is the spiral: not complexity, but the same decision re-implemented in six places with no shared key.

The fix is not to write more RMA logic. It is to stop reshaping the data at each border. Market becomes a first-class record — currency, tax treatment, locale, consent regime, merchant of record, withdrawal terms, disposition rules — and the identifier resolves against it. One thread, annotated by market, rather than one thread per market.

Why this became urgent rather than merely sensible

Those operational numbers used to live in a world with a **reconciliation window**. The warehouse and commerce could disagree until month-end and someone fixed it in a spreadsheet. Nobody was harmed, because nobody asked at event time.

Shopify and Google closed that window. Both moved the trust boundary server-side — Consent Mode v2, Merchant Center matching, the Shopping Graph entity. Commerce now resolves *at the event*. The warehouse plane still resolves *nightly*. A divergence that used to be a month-end chore is now a live inconsistency between what is advertised, what is on the shelf, and what the firmware reports.

Then the obligations land on exactly that seam:

- **CRA Article 14**, from 11 September 2026, obliges reporting of actively exploited vulnerabilities — including for products already on the market. Scoping that report requires firmware ↔ serial ↔ GTIN ↔ shipment ↔ customer. Break one link and you cannot answer.
- **The EU Omnibus price rules** require a 30-day prior-lowest reference, which must attach to the same entity the ads bid on, or your evidence is about a different product than your promotion.
- **Returns and warranty** arrive as a physical code and must resolve to terms, batch and firmware.

The shape changed underneath: parent/child as a graph, not a column

The reconciliation window did not just get shorter. **The thing being reconciled changed shape.**

Shopify's Summer '25 Edition (Horizon, May 2025) shipped a product model that is a graph rather than a table. **Combined Listings** links separate products so they present as variants of one — a genuine parent/child relationship between distinct product records. Add the Standard Product Taxonomy category, options, variant-level identifiers, and metafields, and a "product" is now a nested object with typed relationships, queried through GraphQL and paired with server-side customer events carrying item-level data at event time.

A flat file cannot represent that. It has never been able to. The workaround the channel world settled on is `item_group_id` — every variant gets its own row and a shared string tells the consumer which rows belong together. That is a convention, not a data model: nothing enforces it, nothing validates it, and the relationship exists only in the mind of whatever parses the file.

So the collision is not about latency. It is about **disagreement over what a product is**:

	COMMERCE PLANE	FLAT-FILE PLANE
Product	Nested object with typed children	Rows sharing a string
Identity	Parent and variant both addressable	One row per sellable unit
Relationship	Enforced by the platform	A naming convention
Timing	Resolved at the event	Resolved at the next load

When the WMS, the ERP and customer service all receive the flattened view, they each hold a *keyless approximation* of a structure the commerce plane

holds precisely. Then the questions that matter arrive and resolve differently in each system:

- **A return comes back.** Is it the parent or the child? The customer bought a combined listing; the warehouse holds a discrete SKU; the ERP holds a third record.
- **Inventory is committed** against the row, but sold against the graph.
- **A recall must be scoped.** Which children of which parent, in which markets, on which firmware — a question the flat file cannot express and therefore cannot answer.
- **Price evidence** attaches to whichever record the auditor is shown, and the three do not agree.

This is why the shape question is not academic. **Nesting is not a formatting preference — it is the difference between a relationship the system enforces and one everybody assumes.** The moment the commerce plane started resolving that graph at event time, every downstream system still receiving a flattened export began holding a slightly different product than the one being sold.

The remedy is the same as everywhere else in this document, and it is deliberately unglamorous: keep the flat file as a *transport*, and hold the relationship in a mapping you own that can represent it — parent, child, variant, unit, market. The file delivers. The mapping decides.

The GID rename, which breaks more than people expect

There is a smaller change inside the GraphQL move that causes more integration damage than the big ones, because it looks cosmetic.

REST returned a bare numeric identifier. GraphQL returns a **GID** — a namespaced string:

```
REST      1234567890

GraphQL   gid://shopify/Product/1234567890

          gid://shopify/ProductVariant/9876543210
```

Three consequences, in ascending order of how late you find them:

1. Type changes. Any column, spreadsheet or interface field typed as an integer now receives a string. Some systems coerce, some truncate, some silently drop it. A planning system with a numeric item key does not accept a URI.

2. Joins break quietly. The dangerous state is partial migration — one system stores `1234567890`, another stores `gid://shopify/Product/1234567890`, and the join simply returns nothing. Not an error. An empty result, which reads as "no matching orders" rather than "your key format diverged."

3. Stripping the prefix loses the type, and the type was carrying meaning. The obvious fix is to parse the number back out. But `Product/1234567890` and `ProductVariant/1234567890` occupy separate namespaces — the bare number is ambiguous, and the GID was telling you *which kind of thing* it identified. Discarding it to fit an integer column throws away the one piece of information that made parent and child distinguishable.

That third point is why the GID rename matters for CPG specifically: it lands precisely on the parent/child question. When a combined listing has a parent record and child records, the difference between them is carried in the GID's type segment. Flatten it and the item master can no longer tell you whether a row is the thing you sell or the thing you ship.

Store the full GID. It costs a varchar and it is the difference between a key that means something and a number that used to.

And note the shape of the item master this implies for anyone planning toward 2027 — five identifiers, coexisting, none of them replacing the others:

FIELD	OWNER	PURPOSE
`gid`	Shopify	Commerce identity, typed, parent/child aware
`gtin`	GS1 registry	Global identity, scannable, Digital Link
`mpn`	You	Manufacturing and firmware binding
`planning_item`	ERP / planning estate	Supply chain and pick-pack
`market`	You	Which obligations apply

That table is the whole project. Everything else is deciding where it lives and who may change it.

Every retail road still runs through Google — AI or no AI

It is worth separating this from the AI conversation, because it was true before and stays true regardless.

Whatever mix of channels you sell through — own store, marketplace, retail partner, social — Google is where the product gets **matched to an entity**. The Merchant Center feed resolves your item into the Shopping Graph; Shopping and Performance Max bid against that matched entity; Meta, TikTok and Pinterest catalogs consume the same identifiers; conversions attribute back to it; and AI answer surfaces read the same graph. Adding AI raised the stakes. It did not create the dependency.

Which means the identifier is not only an internal join key. **It is the thing that decides whether your product exists as one entity in the layer most discovery passes through, or as an orphan.** A missing or wrong GTIN does not produce a

clear error — it produces poor matching, and poor matching looks like a bidding problem for months before anyone suspects the data.

Detection that runs in real time, not at month-end

The useful consequence is that this loop can be closed automatically, and most teams do not close it.

- **Merchant Center product status** is queryable per item — disapprovals, warnings, identifier mismatches, price and availability discrepancies against the landing page. That is an API surface, not a dashboard you have to remember to open.
- **The Merchant Center BigQuery transfer** lands product status and performance into your warehouse on a schedule, joinable to your own catalog.
- **GA4's BigQuery export** delivers event-level data with the campaign and item parameters intact.

Put those beside the resolver mapping and you have a genuine detection loop: **the identifier the warehouse holds, the identifier the feed submitted, and the identifier Google matched — compared continuously, with a difference raising an alert rather than waiting for a quarterly audit.**

That is the same instrument-then-automate order as everywhere else in this document. The comparison is worth building even if no agent ever acts on it, because it answers a question nobody can currently answer: *are the physical estate, the commerce estate and the discovery estate talking about the same product right now?*

And once it exists, it is exactly the signal an agent can act on — reconciling a mismatch, re-submitting a corrected item, flagging a batch whose disposition changed. But the detection has to exist first. A loop pointed at an environment

where nobody knows which identifier is authoritative will iterate confidently on the wrong one.

The behemoth, and why you should not touch it

Most of this data currently lives behind a planning estate — Blue Yonder, Manhattan, SAP retail — plus GDSN data pools, supplier onboarding by flat file, and EDI still carrying the volume. Long implementations, change control rather than editing, batch by design, and an item master that quietly became the de facto product truth because the supply chain trusts it more than commerce does. Adding a field is a project.

A note on names, because the rebrands hide the age. The system your team calls JDA is now Blue Yonder: JDA Software acquired the German ML firm Blue Yonder in 2018 and took the acquired company's name for the whole business in **February 2020**; Panasonic then bought a minority stake in 2020 and the remainder in 2021, and it now sits inside Panasonic Connect. The channel platform many teams still call ChannelAdvisor is now **Rithum** — CommerceHub acquired ChannelAdvisor in 2022 and the combined company rebranded in **2024**.

Both are the same manoeuvre: consolidate the incumbents, retire the name that carries the history. It matters here only because a new name invites the assumption of a new architecture, and the installed base is largely the old one. Modernisation is genuinely under way — Blue Yonder has a SaaS re-platform and bought One Network Enterprises and the returns specialist Doddle in 2024 — but the modern platform runs *alongside* the legacy estate rather than retiring it, and migrations are multi-year.

Which is the practical point: **the vendor's own modernisation programme will not reach most customers before your deadlines do.** Plan as though the estate

you have is the estate you will have in September.

Do not migrate it. Map it.

You do not need the planning system to hold the GTIN. You need a mapping you own — planning item ↔ GTIN ↔ commerce variant ↔ firmware revision — resolved server-side at event time, with its own record. The planning estate keeps doing exactly what it does. The resolver absorbs the impedance mismatch, and the batch cadence stops being a correctness problem because the mapping is authoritative even when the nightly run is not.

That is a table and a function, not a re-platforming. It is also the only version a supply-chain owner will agree to, because it asks nothing of them.

The quietly important part: the resolver is yours

GS1 Digital Link changes the identifier from *a number you send to a data pool* into **a URI you host**:

```
https://yourdomain.com/01/09506000134352
```

The GTIN is still GS1's. **The endpoint is yours.** What that URL returns — price history, SBOM, consent notice, verification certificate, localised content per market, return instructions for the market the scan came from — is your data model, on your infrastructure, changeable without a data-pool sync or anyone's certification cycle.

That is the first genuinely web-native thing in this stack, and it routes around the flat-file world without confronting it. The scanner still gets a number. Everything else stops being a CSV.

Who gets to change it

The last piece is governance, and it decides whether any of this survives contact with an organisation.

If a business analyst needs to add a country code — one field that propagates to WMS, ERP, RMA and customer service, and the location-based funnel — that change should be made by the person who **owns the revenue outcome in that market**, under a bounded mandate: scoped, capped, expiring, revocable, and recorded. Not raised as a ticket to a team ten timezones away with a change window.

The usual objection is governance: *who is accountable, where is change control.* It deserves a straight answer, because in the old architecture the objection was correct — unbounded access with no record genuinely was reckless, and the queue was the only control available.

But a queue was never actually a control. **Whoever picks up a ticket holds full access, and the only limit is what they chose to type.** A mandate is scoped, capped, expiring, revocable, and records what was refused as well as what was done. The objection is right about the risk and wrong about which option carries it.

What to do before you have GTINs

You are not blocked. In order:

0. **First, check whether you already have them.** If you sell through any big-box retailer, you do — inside the channel management platform. Recovering that mapping into a system you control is faster than licensing afresh, and it is the step most enterprises skip because they assume the identifier is missing rather than merely rented.

1. **If you genuinely have none, start the GS1 licence.** It is the long pole and it is commercial.
2. **Make market a first-class record** — not a display preference. This is the work that pays off regardless of Sunrise.
3. **Bind every identifier you already have** — MPN, SKU, planning item — to a single product entity now, so the join is correct on the day GTINs arrive rather than being retrofitted.
4. **Stand up the resolver** at `/01/…`, even returning a stub. The route existing is what lets everything else be tested.
5. **Instrument the thread** — factory, warehouse, sale, return — so you can answer *which units* before anyone asks.
6. **Artwork last.** It is the only step that cannot be changed after it ships.

Honest disclosure

At the time of writing, the catalog behind this document carries `mpn` and `sku` on every product and **no GTINs at all**, and the `/01/…` resolver is not built. The QR pipeline exists and every minted code carries its product identifier; the GTIN does not exist yet, and a Digital Link cannot be minted without one.

That is the ordinary starting position, and it is stated here because a readiness article written by someone pretending to be finished is not worth reading.

The encouraging part, which is easy to miss

None of this requires the legacy estate to change, and that is worth ending on.

Commerce was always server-side. Catalogs, orders, inventory, pricing — all of it resolved on a server long before anyone said "server-side" as a strategy. The

past decade of client-side tags and browser-resident tracking was the aberration, not the baseline. What has happened recently is a return: Google moved its consent and measurement boundary back to the server, and **Shopify joined it**, resolving identity, consent and catalog at the event rather than in the page.

Which means there is now a **real-time plane above the batch estate** — and you do not have to earn your way onto it by first fixing everything underneath.

That is why LLM-based and agentic commerce is achievable in an enterprise that still runs planning on a nightly cycle and still receives a supplier CSV on Tuesdays. The agent does not query the planning system. It calls a bounded server-side function, which resolves against a mapping you own, which was *fed* by the CSV but is not *arbitrated* by it. The flat file becomes a source among sources rather than the thing that decides what is true.

Stated plainly:

- **The CSV stays.** It is a delivery mechanism, and there is nothing wrong with a delivery mechanism.
- **The planning estate stays.** It keeps planning. Nobody has to justify a migration.
- **The resolver is the join**, holding the identifier mapping and answering at event time, with a record.
- **Agents talk to the resolver**, never to the legacy directly, so their authority is bounded and their actions are attributable.

The batch cadence stops being a correctness problem the moment something authoritative sits above it. That is the whole trick, and it is a smaller piece of work than the size of the surrounding systems suggests.

Sunrise 2027 is a retailer capability date. The work is a data-model change. And the identifier you assign this year decides whether a scan in 2028 resolves to one product — or to nothing.

CRM Sync · Webflow · Xano · Cloudflare · Shopify

Monochrome · print / PDF ready